LAB NOTEBOOK ▪ 2026-10-20 ▪ 6 min read

An Agent Lab for Scientific Software Optimization

One scientist, one agent, one deforming bubble: a session that took a C++/MPI solver from FP64 to "ready to evolve" in an afternoon, and what it cost.

95 min	18 min	$7.59	1.0
session	of scientist time	in tokens	core-hour of emulation

The ask

The lab is built to lower the precision of scientific codes, safely. The code under study sits in app/ and changes with each scientist; everything around it is general. Today's app is the Deforming Bubble, a level-set mini-app ported from Flash-X (WENO5 advection, redistance, MPI), standing in for the DOE applications the lab is meant for. The scientist dropped it in and typed:

> *I have added my code to the app, and ready for optimization.*

The agent's first move was a polite *not yet*. Optimization evolves a low-precision *seed*, and no seed existed. So it read the lab's rulebook, checked the app against its 13-point contract (all pass, natively and in Docker), and proposed the first stage instead: *raptorize*. Fourteen minutes after the ask, a plan was waiting, with every decision filled in and its source cited. The scientist didn't have to answer a single question to get there.

Meet the lab

```
MxP-Agent-Lab/
├── app/ ..... swappable: any code meeting app.md (today: the bubble)
├── dev/workflows/ ..... general: the lab's playbook
│   ├── protocol.md ..... plan mode, run mode, checkpoints
│   └── raptorize/ optimize/ verify/ ..... one contract per stage
└── experiments/2026-10-20/ ..... today's question
    ├── README.md ..... one row per run, with its commit
    ├── 1-raptorize/ ..... plan.md, raptorize.patch, results/
    ├── 2-optimize/ ..... plan.md, experiment.yaml, evaluator.py
    └── report.tex, session.py ..... this post, and its numbers
```

Swap app/ and the rest stays. Stages run **raptorize** → **optimize** → **verify**. Each stage is a contract in `dev/workflows/`; each run of it gets a folder in `experiments/`. Every run starts with a *plan*, then executes it, stopping at checkpoints where only the scientist can say go.

Figure 1: The whole lab on one screen: the app changes, the lab doesn't.

How it went

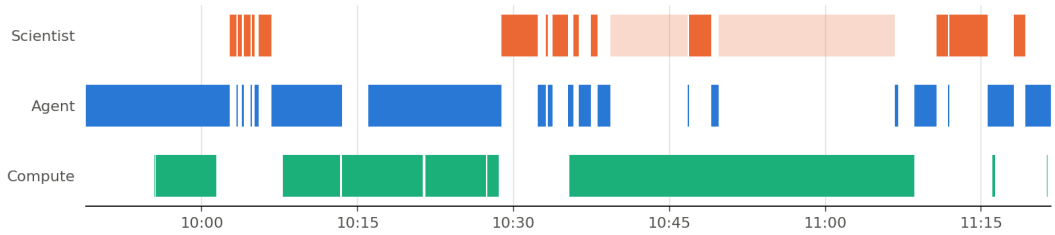


Figure 2: Who was busy when. Pale orange: the scientist was free while runs went. The scientist's lane adds up to 18 minutes.

Seventeen messages, 282 words from the scientist. The best moments came from the scientist pushing back:

"Why did you choose advect.cpp?" The honest answer was "I read the code." Two minutes later there was a profile: advection 52% of runtime, redistance 24%. It also turned up a wrong number in the plan, which got fixed.

"Was your decision influenced by a memory?" No. It came from the stage's default ("the hottest loop") and the code. A plain question, a plain answer.

"Run everything inside the container." The agent re-ran the FP64 baseline in the container too, so "unchanged" compares one compiler with itself, not two.

"Can I look at some plots?" At 100 steps the four bubble outlines were identical. Unremarkable, so the scientist asked to *run until the bubble deforms*.

"The runs still working?" Yes, but at a third of the expected speed. OpenMPI had pinned four jobs onto two cores. One flag fixed it.

"Why not 2–3 MPI ranks?" Three doesn't divide a 128-cell grid; two does, and the field came out bit-identical to the single-rank run in all 11 frames.

"What do you mean, checkpoint 3?" The one miss. The "record and commit?" question was buried under three follow-up requests, and had to be explained again.

What came out

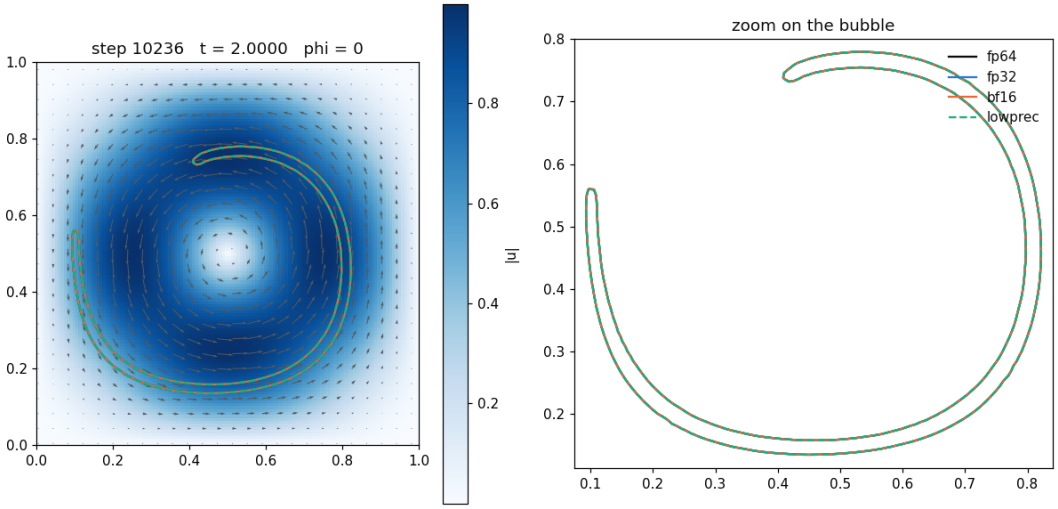


Figure 3: The bubble at $t = 2$, after 10 236 steps, in four precisions. Spot the difference: there isn't one at this scale.

mode	RMS error at $t = 2$	the short story
fp32 (RAPTOR emulated)	6.3×10^{-7}	30 min of emulation
float seed (native)	6.3×10^{-7}	same answer, bit for bit, in 17 s
bf16 (RAPTOR emulated)	2.6×10^{-4}	400× worse, still invisible

The float seed matching emulated fp32 bit for bit is the best possible check that it's a faithful copy. The error maps showed the error lives far from the bubble's edge, where it barely matters. That's a hint the optimize score should look near the interface.

Optimize, so far. The plan, evaluator and job are written. The seed scored 9.2722 twice (the same both times). The launch waits on Google Cloud credentials.

What it cost

Claude Opus 5.5, 103 calls	tokens	$/M	$
fresh input	208	4.00	0.00
cache writes	624 107	5.00	3.12
cache reads	13 294 158	0.20	2.66
output	90 268	20.00	1.81
total			**7.59**

95% of what the model read came from the cache, at a twentieth of the price. The surprise is that cache *writes* are the biggest line. The cache lives five minutes, and this lab waits: a half-hour run, a scientist looking at plots. Every long pause meant re-caching the whole conversation. A one-hour cache would fit this lab better.

Human time vs. time saved

Doing it by hand (our estimate, for an expert new to the lab)	
read the lab rules and the app, pick and profile the kernel	1.5 h
precision modes, shadow run, error columns, RAPTOR build	4 h
gate, plots, full runs (and spotting the core pinning)	1.5 h
optimize plan, evaluator, calibration	1.5 h
write this up with figures and costs	2 h
total	**≈ 10 h**
With the agent: scientist's attention	**18 min**

Roughly a 30× cut in human time, for $7.59. Because only app/ is specific, the next DOE code reuses all of it: the contracts, the checks, and this report's script. The agent was working for 49 of the 95 minutes. The scientist spent their time on what only a scientist can do: questioning choices, asking for the run that matters, and widening the test.

What we'd change

- **Keep open gates in view.** A pending checkpoint shouldn't scroll away.
- **Cache for the lab's pace:** a one-hour cache when runs take half an hour.
- **Score where the physics is:** an error measured near the interface for optimize.
- **Small papercut:** the app should create its output folder's parents.

Numbers: `session.py` reads the session transcript (token usage per call, timestamps, tool calls). Runs, plans and patches are under `experiments/2026-10-20/`.